



Booster Challenge 2026

Violintonate

Author: Dávid Novák
Email: xnovakd00@stud.fit.vutbr.cz
Faculty: VUT FIT

September 5, 2026

1 Outcome

Program is able to estimate rhythmical and intonation accuracy of a violinist based on homemade sound recording and sheet music in MusicXML format. The output is a text table and a graph with information determining repertoire interpretation accuracy.

Measure	:	10		
Note	:	F5		
Rhythm	:	Onset	Offset	
Time difference	:	-0.05	+0.08	
Beats difference	:	-0.07	+0.10	
Evaluation	:	perfect	perfect	
Intonation	:	Perfect	Okay	Wrong
Start	:	98%	2%	0%
Middle	:	99%	1%	0%
End	:	95%	5%	0%

Figure 1: CLI Output

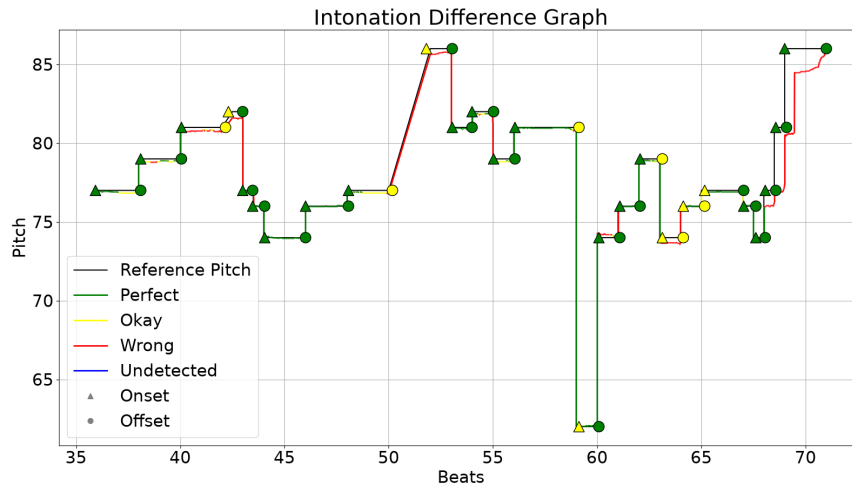


Figure 2: Intonation Difference Graph

The user is able to set these parameters:

- the number of a part if the score contains multiple instruments
- time signature (4/4, 3/4, 6/8, etc.)

- start and end measure and their offsets in beats
- tempo in beats per minute
- switching between text table output and .json output with onset and offset times
- possibility to open an interactive graph or to save it

2 Technical Details

2.1 MusicXML Parser

MusicXML file with sheet music is parsed using music21 python library [1]. It stores score events into list of dictionaries which contains information about event kind (note/rest) [2], measure number, pitch frequency value in midi and start and end quarter length (conversion to seconds depends on the tempo's beat unit, 4/4, 6/8, etc.) of each score event.

2.2 Score Time Mapper

Score Time Mapper operates on score events data, bpm, time signature, start measure, start measure offset in beats, end measure and end measure offset in beats. It crops the unnecessary score events and adds start and end times in seconds relative to the audio recording with expected one bar count in.

2.3 Pitch Extractor

Based on the start and end second of the audio computed by the Score Time Mapper, Pitch Extractor extracts frames containing pitches and RMS values of audio recording using Librosa python library. It extracts pitches using pYIN algorithm returning not only fundamental frequencies of the frames, but also pitch confidence probability flags and values [3].

The uncertain frames are filtered later. For easier work with data, pitch values are converted into fractional MIDI values.

RMS values are computed standardly by quadratic mean using Librosa's built in function [4]. Every frame has a corresponding time of the audio recording in seconds [5].

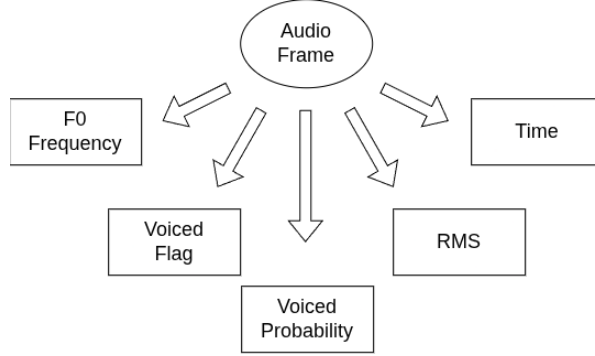


Figure 3: Audio Frame Extraction

2.4 Voiced Pitch Filter

Not every frame can be used for further analysis, because of the potential white noise or uncertain pitch frames, such as a bow scratches or attacks. That is why the frames are filtered by their voiced flags and voiced probabilities.

$F = \{f_1, f_2, \dots, f_n\}$	set of audio frames sorted by time,
$V(f) \in \{\mathbf{false}, \mathbf{true}\}$	voiced flag of frame f ,
$p_v(f) \in [0, 1]$	voiced probability of frame f ,
$p_{th} = 0.5$	voiced probability threshold,
$A \subseteq F$	set of frames used for further analysis.

$$f_i \in A \iff (V(f_i) = \mathbf{true} \wedge p_v(f_i) \geq p_{th})$$

2.5 RMS Onset Offset Detector

Before we start with RMS onset and offset detection, we smoothen up the signal to prevent false local minima localization. That is done by block-average smoothing.

$R_{all} = \{r_1, r_2, \dots, r_n\}$	set of all RMS values,
$R_m = \{m_1, m_2, \dots, m_k\}$	set of smoothened RMS values,
$T = \{t_1, t_2, \dots, t_n\}$	measured times of frames in set A ,
$T_m = \{t_{m_1}, t_{m_2}, \dots, t_{m_k}\}$,	set of smoothened RMS frame times.

$$m_k = \frac{1}{5} \sum_{i=5k-4}^{5k} r_i; \quad t_{m_k} = \frac{1}{5} \sum_{i=5k-4}^{5k} t_i$$

Expected onset times are score events' start second values. Expected offset times are score events' end second values. Firstly, we determine the potential onset and offset location windows based on the note boundary constant N_b .

$$\begin{aligned}
T_{\text{on}} &= \{t_{\text{on}_1}, t_{\text{on}_2}, \dots, t_{\text{on}_n}\}, & \text{set of expected onset times,} \\
T_{\text{off}} &= \{t_{\text{off}_1}, t_{\text{off}_2}, \dots, t_{\text{off}_n}\}, & \text{set of expected offset times,} \\
\hat{T}_{\text{on}} &= \{\hat{t}_{\text{on}_1}, \hat{t}_{\text{on}_2}, \dots, \hat{t}_{\text{on}_n}\}, & \text{set of detected onset times,} \\
\hat{T}_{\text{off}} &= \{\hat{t}_{\text{off}_1}, \hat{t}_{\text{off}_2}, \dots, \hat{t}_{\text{off}_n}\}, & \text{set of detected offset times,} \\
N_b &= 0.15, & \text{note-boundary constant in seconds,} \\
W_{\text{on}_i} &= [t_{\text{on}_i} - N_b, t_{\text{on}_i} + N_b], & \text{onset location window,} \\
W_{\text{off}_i} &= [t_{\text{off}_i} - N_b, t_{\text{off}_i} + N_b], & \text{offset location window,} \\
f_m : T_m &\rightarrow R_m, \quad f_m(t_{m_i}) = m_i, & \text{RMS-at-time function.}
\end{aligned}$$

Now, when we know the onset and offset location windows, the RMS-based onset and offset detection works by finding rightmost local minima nearby expected onsets and leftmost local minima nearby expected offsets:

$$\begin{aligned}
\hat{t}_{\text{on}_i} &= \max \{ \tau \in W_{\text{on}_i} \mid \tau \text{ is a local-minimum location of } f_m \}, \\
\hat{t}_{\text{off}_i} &= \min \{ \tau \in W_{\text{off}_i} \mid \tau \text{ is a local-minimum location of } f_m \}.
\end{aligned}$$

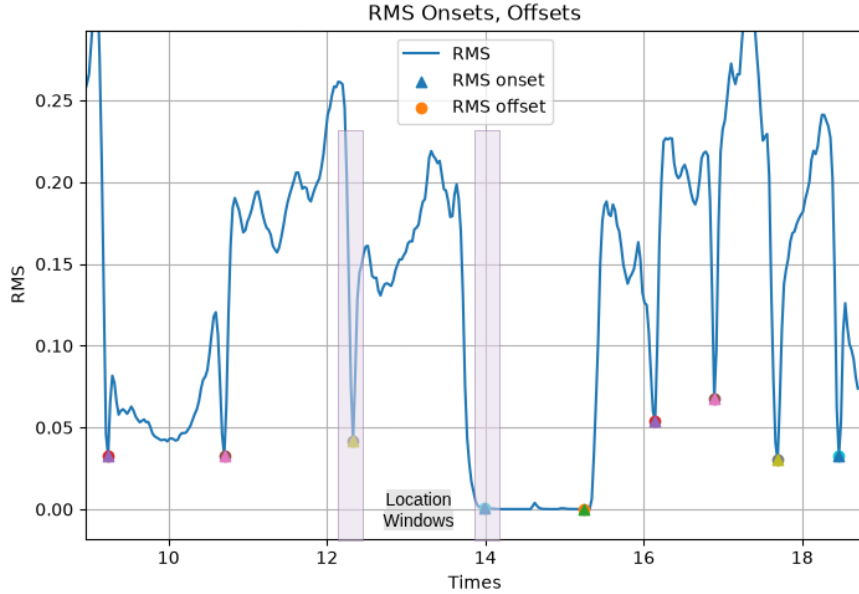


Figure 4: RMS Onsets and Offsets

2.6 Pitch Change Detector

Inside offset location windows, which were defined previously, we need to locate pitch transition points. We only localize consecutive score events, where one note transitions into another ($event_i$ and $event_{i+1}$ kind is a *note*), so onset location windows are not necessary.

First, we have to locate the current and next pitch of the score event, which are the first n pitch frames outside of the potential onset and offset location window to the right and to the left. Number of frames for analysis on each side is selected by the *rolling median count constant* M .

$$\begin{aligned}
 M = 7 & && \text{rolling median count constant,} \\
 P = \{p_1, p_2, \dots, p_n\}; & && \text{measured pitches of frames in set } A, \\
 l = \min\{i \mid t_i \in W_{off}\} & && \text{first pitch time index in } W_{off}, \\
 r = \max\{i \mid t_i \in W_{off}\} & && \text{last pitch time index in } W_{off},
 \end{aligned}$$

$$f_p : T \rightarrow P, f(t_i) = p_i \quad \text{pitch-at-time function,}$$

$$\begin{aligned}
 P_p &= \{f(t_{l-M}), f(t_{l-M+1}), \dots, f(t_{l-1})\} && \text{set of previous pitches,} \\
 P_n &= \{f(t_{r+1}), f(t_{r+2}), \dots, f(t_{r+M})\} && \text{set of next pitches.}
 \end{aligned}$$

The previous and next pitch sequence is first smoothed using a rolling median filter with a window size of 3.

$$\begin{aligned}
 P'_p &= \{p_{l-M}, p_{l-M+1}, \dots, p_{l-1}\} && \text{previous rolling-median pitches,} \\
 P'_n &= \{p_{r+1}, p_{r+2}, \dots, p_{r+M}\} && \text{next rolling-median pitches.}
 \end{aligned}$$

Then take the median value of the rolling median. This makes the final estimate less sensitive to unusually high or low values.

$$\begin{aligned}
 u &= \text{median}(P'_p) && \text{previous pitch,} \\
 v &= \text{median}(P'_n) && \text{next pitch.}
 \end{aligned}$$

Now, that we know previous and next pitch of the score event, we can evaluate the pitch transition times of the frames.

S_p constant (same pitch midi tolerance in fractional midi) will tell, if the pitch of the frame is sufficiently close for it to be called the same pitch:

$$p_1 \approx p_2 \iff |p_1 - p_2| < S_p; \quad p_1, p_2 \in P; \quad S_p = 0.5$$

Then we find the transition times:

$$\{\hat{t}_{tr_1}, \hat{t}_{tr_2}, \dots, \hat{t}_{tr_n}\} \in \hat{T}_{tr}; \quad \text{where } \hat{T}_{tr} \text{ is a set of detected transition times.}$$

$$\hat{t}_{tr_j} = \min \{t_i \mid l < i < r \wedge f(t_i) \approx v_j \wedge f(t_{i+1}) \approx v_j \wedge f(t_{i+2}) \approx v_j\}.$$

If three consecutive pitches inside the offset location window are the same, then we found our transition point.

2.7 Rhythm Analyzer

There are two approaches to detecting note transitions: identifying rhythmic boundaries using RMS-based onsets and offsets, or detecting pitch transition points.

Based on previous research, the detection is more accurate using pitch transitions.[6] We use RMS-based onsets and offsets for the same consecutive notes.

$$\begin{aligned} D_p &= \{d_{p_1}, d_{p_2}, \dots, d_{p_n}\} && \text{set of pitch transition diffs,} \\ D_{on} &= \{d_{on_1}, d_{on_2}, \dots, d_{on_n}\} && \text{set of onset diffs,} \\ D_{off} &= \{d_{off_1}, d_{off_2}, d_{off_n}\} && \text{set of offset diffs.} \end{aligned}$$

We compute the time diffs in seconds for further intonation analysis and in beats for user-friendly output.

If there is a note pitch transition, then $t_{off_i} = t_{on_{i+1}}$, so let the expected transition times be the expected offset times, where the pitch transitions occur:

$$\begin{aligned} T_{tr} &\subseteq T_{off} \\ d_{p_i} &= \hat{t}_{tr_i} - t_{tr_i}; \quad d_{p_i} \in D_p, \hat{t}_{tr_i} \in \hat{T}_{tr}; t_{tr_i} \in T_{tr} \end{aligned}$$

else, compute the difference based on RMS-based onsets and offsets:

$$\begin{aligned} d_{on_i} &= \hat{t}_{on_i} - t_{on_i}; && d_{on_i} \in D_{on}, \hat{t}_{on_i} \in \hat{T}_{on}, t_{on_i} \in T_{on}, \\ d_{off_i} &= \hat{t}_{off_i} - t_{off_i}; && d_{off_i} \in D_{off}, \hat{t}_{off_i} \in \hat{T}_{off}, t_{off_i} \in T_{off}. \end{aligned}$$

Now we compute player's interpretation times:

$$\begin{aligned} T_{tr}^* &= \{t_{tr_1}, t_{tr_2}, \dots, t_{tr_n}\} && \text{set of player's interpretation transition times,} \\ T_{on}^* &= \{t_{on_1}, t_{on_2}, \dots, t_{on_n}\} && \text{set of player's interpretation onsets,} \\ T_{off}^* &= \{t_{off_1}, t_{off_2}, \dots, t_{off_n}\} && \text{set of player's interpretation offsets.} \end{aligned}$$

$$\begin{aligned} t_{tr_i} &= \hat{t}_{tr_i} + d_{p_i}; && t_{tr_i} \in T_{tr}^*, \hat{t}_{tr_i} \in \hat{T}_{tr}, d_{p_i} \in D_p \\ t_{on_i} &= \hat{t}_{on_i} + d_{on_i}; && t_{on_i} \in T_{on}^*, \hat{t}_{on_i} \in \hat{T}_{on}, d_{on_i} \in D_{on} \\ t_{off_i} &= \hat{t}_{off_i} + d_{off_i}; && t_{off_i} \in T_{off}^*, \hat{t}_{off_i} \in \hat{T}_{off}, d_{off_i} \in D_{off} \end{aligned}$$

In the output, the user gets a flag based on difference in beats.

$$\Delta t_i = |t_i - t_{e_i}|; \quad t_i \in T_{tr}^* \cup T_{on}^* \cup T_{off}^*; \quad t_{e_i} \in T_{tr} \cup T_{on} \cup T_{off}$$

$$\gamma = \text{BEAT_DIFF_TOLERANCE} = 0.10$$

$$\text{rhythm}(\Delta t_i) = \begin{cases} \text{perfect,} & \Delta t_i < \gamma, \\ \text{okay,} & \tau \leq \Delta t_i < 2\gamma, \\ \text{wrong,} & \Delta t_i \geq 2\gamma. \end{cases}$$

2.8 Intonation Analyzer

Firstly, we will calculate intonation analysis windows:

$W_I = \{[a_1, b_1), [a_2, b_2), \dots, [a_n, b_n]\}$ set of intonation windows, where
 a_i and b_i represent onset and offset times.

If there is a note transition between b_i and a_{i+1} :

$$b_i = a_{i+1} = t_{tr_i}; \quad t_{tr_i} \in T_{tr}^*$$

Else, determine by RMS offsets and onsets:

$$\begin{aligned} a_i &= t_{on_i}; & t_{on_i} &\in T_{on}^* \\ b_i &= t_{off_i}; & t_{off_i} &\in T_{off}^* \end{aligned}$$

After that, we take each pitch frame with corresponding time from the intonation window and compute the difference Δp :

$$P_e = \{p_{e_1}, p_{e_2}, \dots, p_{e_n}\} \quad \text{expected pitches from score events}$$

$$\Delta p_i = |p_i - p_{e_i}|; \quad f(t_i) = p_i \wedge t_i \in W_{I_i}; \quad t_i \in T$$

For the user output, the program divides each note to start section, middle section and end section by the ratio $[0 - 0.25); [0.25 - 0.75); [0.75 - 1]$ to be more informative for the user. Intonation tolerance flags are set for each pitch frame accordingly:

$$\tau = \text{INTONATION_TOLERANCE_MIDI} = 0.10.$$

$$\text{intonation}(\Delta p_i) = \begin{cases} \text{perfect,} & \Delta p_i < \tau, \\ \text{okay,} & \tau \leq \Delta p_i < 2\tau, \\ \text{wrong,} & \Delta p_i \geq 2\tau. \end{cases}$$

Then, the user gets a percentage of perfect, okay and wrong pitch frames in each section using the formula:

$$flag\ ratio = concrete\ flag\ cnt \div all\ flag\ cnt$$

Overall flag is calculated this way:

$$flag\ ratio = \begin{cases} \text{wrong,} & \text{wrong ratio} \geq 0.25, \\ \text{perfect,} & \text{perfect ratio} \geq 0.75 \wedge \text{wrong ratio} \leq 0.05, \\ \text{okay,} & \text{otherwise.} \end{cases}$$

3 Experiment

Output of the program was tested using Audacity[7], which cropped the recording based on onset and offset times and was verified by listening to the cropped notes.

Recordings tested various techniques (legato, détaché) with various tempo in scales practice and one practical recording of desired repertoire.

Program can successfully detect the rhythm and intonation differences, if the player can play using a metronome with reasonable tempo accuracy not deviating away from onset and offset location windows and the tempo is not too fast (120 bpm, sixteenth notes tested). Further work needed for fast playing.

4 Further Work

4.1 Backend

- real time feedback
- vibrato detection (pitch oscillation)
- rubato (expressive tempo with no stable bpm), fast tempo pitch recognition
- ornamentation detection and feedback (trills, turns, glissando, accents, ...)
- sheet music in PDF format

4.2 GUI

- sheet music rendering with intuitive intonation and rhythm feedback
- user-friendly graphical interface

5 Use/deployment in practice

The application could be used as a practice tool for violin students, helping them with rhythm and intonation. A user provides a homemade recording and a score of the repertoire. The program analyzes and compares the recording with the sheet music and displays the feedback.

Currently, the prototype works as a command line interface tool comparing the rhythm and intonation of player’s interpretation with sheet music file in musicXML format. Later versions could integrate the analysis backend into a graphical user interface with sheet music rendering as a desktop or smartphone application.

The application could be desirable for self-taught players or teachers to help their students practice at home.

References

- [1] music21 Development Team, *music21.converter – File and data conversion*, music21 documentation, <https://music21.org/music21docs/moduleReference/moduleConverter.html>, accessed August 25, 2026.
- [2] music21 Development Team, *music21.note – Notes, rests, and lyrics*, music21 documentation, <https://music21.org/music21docs/moduleReference/moduleNote.html>, accessed August 25, 2026.
- [3] librosa Development Team, *librosa.pyin – Fundamental frequency estimation using probabilistic YIN*, librosa documentation, <https://librosa.org/doc/0.11.0/generated/librosa.pyin.html>, accessed August 25, 2026.
- [4] librosa Development Team, *librosa.feature.rms – Root-mean-square (RMS) computation*, librosa documentation, <https://librosa.org/doc/0.11.0/generated/librosa.feature.rms.html>, accessed August 25, 2026.
- [5] librosa Development Team, *librosa.times_like – Generate time values corresponding to a feature matrix*, librosa documentation, https://librosa.org/doc/0.11.0/generated/librosa.times_like.html, accessed August 25, 2026.
- [6] Che-Yuan Liang, Li Su, Yi-Hsuan Yang, and Hsin-Ming Lin, “Musical Offset Detection of Pitched Instruments: The Case of Violin,” *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR)*, Málaga, Spain, pp. 281–287, 2015.
- [7] Audacity Team, “Audacity,” software, version 3.4.2. [Online]. Available: <https://www.audacityteam.org/>